Aim:

Implement Ant colony optimization by solving the Traveling salesman problem using python

Problem Statement:

A salesman needs to visit a set of cities exactly once and return to the original city. The task is to find the shortest possible route that the salesman can take to visit all the cities and return to the starting city.

In [1]:
```
import numpy as np
import random
```

In [2]:
```
# Distance matrix (example: 5 cities)
dist = np.array([
    [0, 2, 2, 5, 7],
    [2, 0, 4, 8, 2],
    [2, 4, 0, 1, 3],
    [5, 8, 1, 0, 2],
    [7, 2, 3, 2, 0]
])
```

In [3]:
```
num_cities = len(dist)
pheromone = np.ones((num_cities, num_cities))
```

In [4]:
```
alpha = 1      # pheromone importance
beta = 2       # distance importance
evaporation = 0.5
num_ants = 5
iterations = 20

best_path = None
best_cost = float('inf')
```

In [5]:
```
for _ in range(iterations):
    all_paths = []

    for _ in range(num_ants):
        visited = [False] * num_cities
        path = [random.randint(0, num_cities - 1)]
        visited[path[0]] = True

        for _ in range(num_cities - 1):
            current = path[-1]
            probs = []

            for j in range(num_cities):
                if not visited[j]:
                    prob = (pheromone[current][j] ** alpha) * ((1 / dist[current][j]) ** beta)
                    probs.append((j, prob))

            total = sum(p for _, p in probs)
            probs = [(j, p / total) for j, p in probs]

            r = random.random()
            cumulative = 0
            for j, p in probs:
                cumulative += p
                if r <= cumulative:
                    next_city = j
                    break

            path.append(next_city)
            visited[next_city] = True

        # Complete cycle
        path.append(path[0])

        # Cost
        cost = sum(dist[path[i]][path[i+1]] for i in range(len(path)-1))

        all_paths.append((path, cost))

        if cost < best_cost:
            best_cost = cost
```

```
            best_path = path

    # Evaporation
    pheromone *= (1 - evaporation)

    # Update pheromones
    for path, cost in all_paths:
        for i in range(len(path)-1):
            pheromone[path[i]][path[i+1]] += 1 / cost
```

In [6]:
```
print("Best Path:", best_path)
print("Best Cost:", best_cost)
```

```
Best Path: [4, 3, 2, 0, 1, 4]
Best Cost: 9
```